COS30017. SOFTWARE DEVELOPMENT FOR MOBILE DEVICES

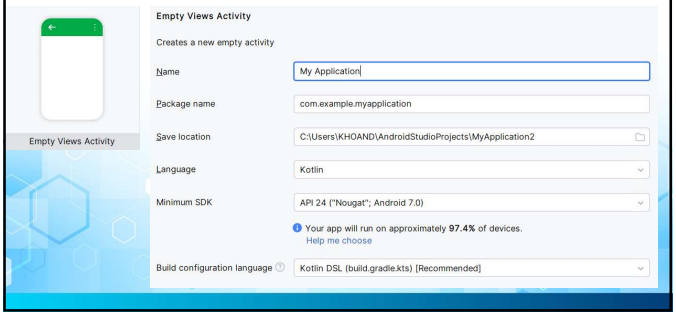
Dr. Nguyen Dang Khoa
Email: kdnguyen@swin.edu.au
Phone: 0974.844.618

Week 2 - Kotlin

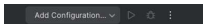
- Does not need to install API 35
- …
- …



Week 2 - Kotlin



Week 2 - Kotlin

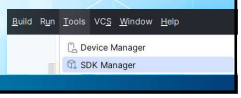


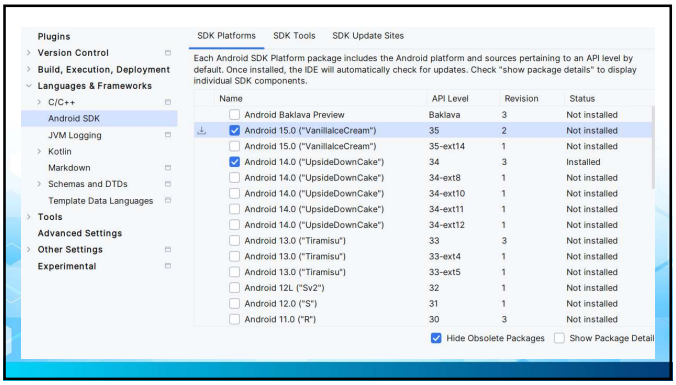
- Click Add Configuration to add Emulator (Pixel 6 API 35)

```
Recommended action: Update this project to use a newer compileSdk
of at least 35, for example 35.

Note that updating a library or application's compileSdk (which
allows newer APIs to be used) can be done separately from updating
targetSdk (which opts the app in to new runtime behavior) and
minSdk (which determines which devices the app can be installed
on).
Update minCompileSdk in modules with dependencies that require a higher minCompileSdk.
```

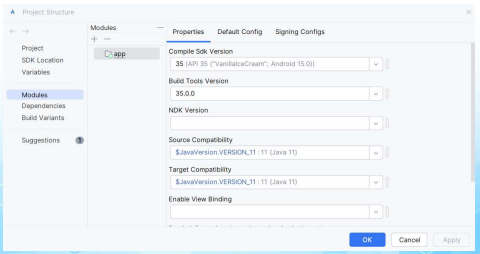
- Tools/SDK Manager

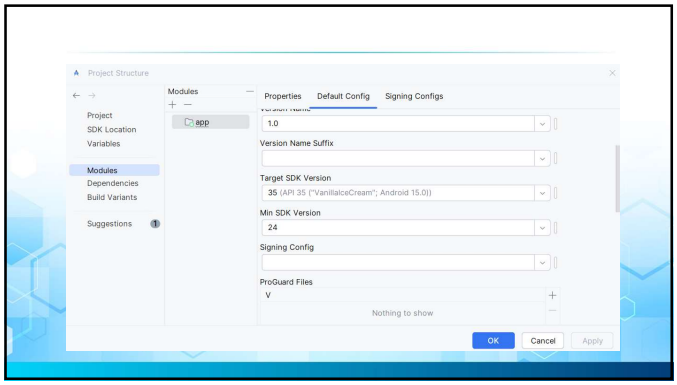


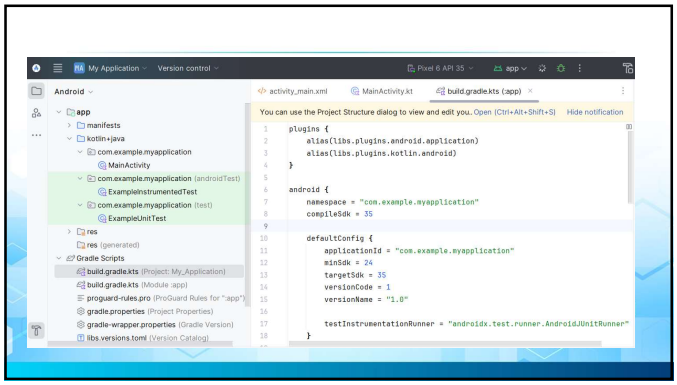


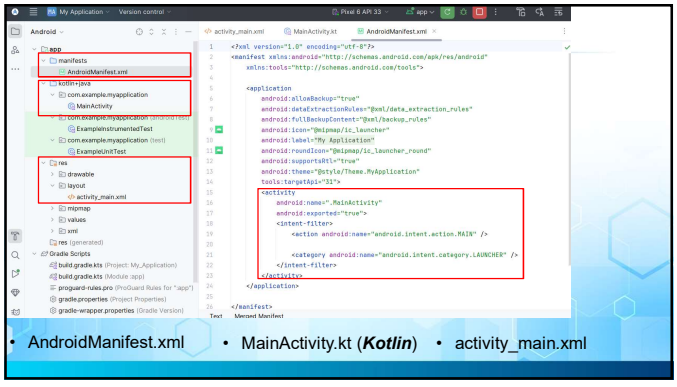
Week 2 - Kotlin

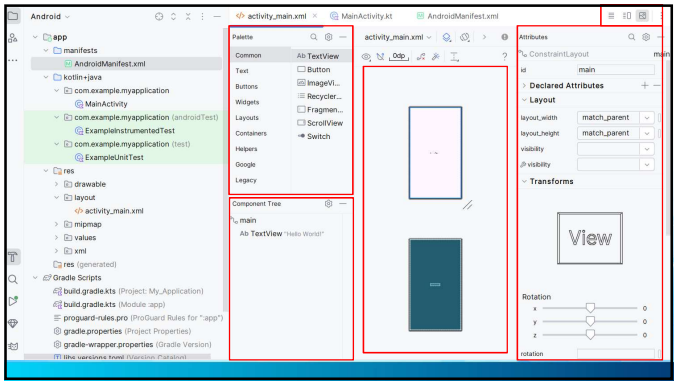
- File/Project structure



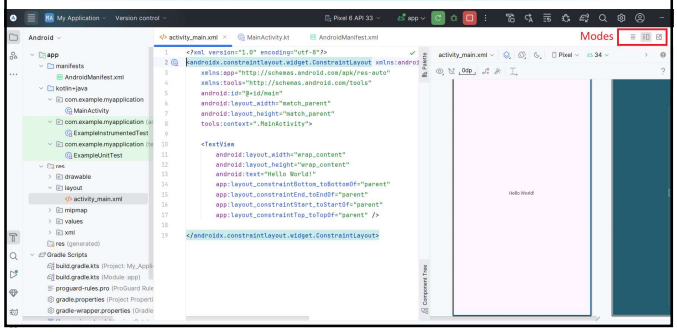

Project Structure
Modules
Properties
Default Config
Signing Configs
Project
SDK Location
Variables
Modules
Dependencies
Build Variants
Suggestions
app
1.0
Version Name Suffix
Target SDK Version
35 (API 35 ("VanillaIceCream"; Android 15.0))
Min SDK Version
24
Signing Config
ProGuard Files
Nothing to show
OK
Cancel
Apply


My Application
Version control
Android
app
manifests
kotlin+java
com.example.myapplication
MainActivity
ExampleInstrumentedTest
ExampleUnitTest
res
Gradle Scripts
build.gradle.kts (Project: My_Application)
build.gradle.kts (Module :app)
proguard-rules.pro
gradle.properties
gradle-wrapper.properties
libs.versions.toml
activity_main.xml
MainActivity.kt
build.gradle.kts (:app)
plugins {
alias(libs.plugins.android.application)
alias(libs.plugins.kotlin.android)
}
android {
namespace = "com.example.myapplication"
compileSdk = 35
defaultConfig {
applicationId = "com.example.myapplication"
minSdk = 24
targetSdk = 35
versionCode = 1
versionName = "1.0"
testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"

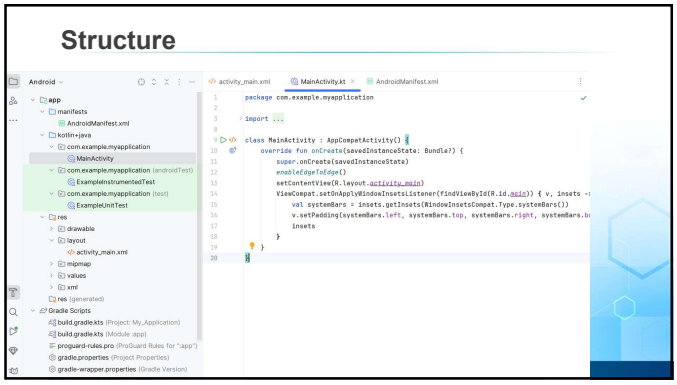

AndroidManifest.xml
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:tools="http://schemas.android.com/tools">
<application
android:allowBackup="true"
android:dataExtractionRules="@xml/data_extraction_rules"
android:fullBackupContent="@xml/backup_rules"
android:icon="@mipmap/ic_launcher"
android:label="My Application"
android:roundIcon="@mipmap/ic_launcher_round"
android:supportsRtl="true"
android:theme="@style/Theme.MyApplication"
tools:targetApi="31">
<activity
android:name=".MainActivity"
android:exported="true">
<intent-filter>
<action android:name="android.intent.action.MAIN" />
<category android:name="android.intent.category.LAUNCHER" />
</intent-filter>
</activity>
</application>
</manifest>
• AndroidManifest.xml • MainActivity.kt (**Kotlin**) • activity_main.xml



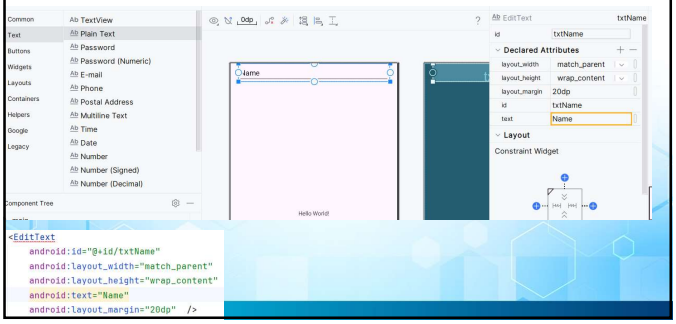
Structure



Structure

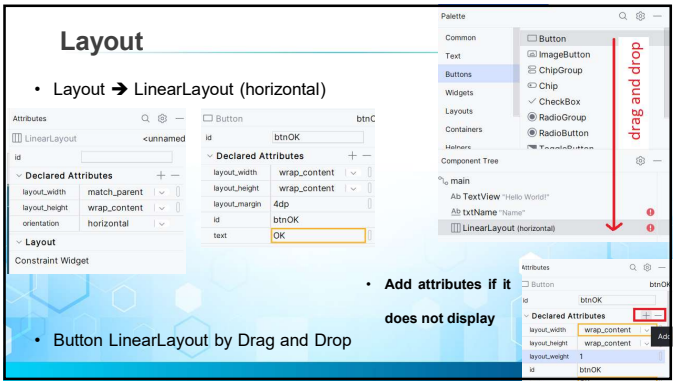


Define one toolbox



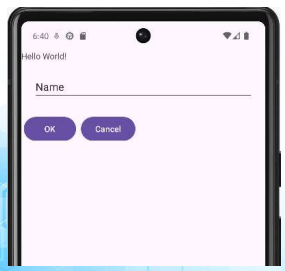
Layout

- Layout ➔ LinearLayout (horizontal)

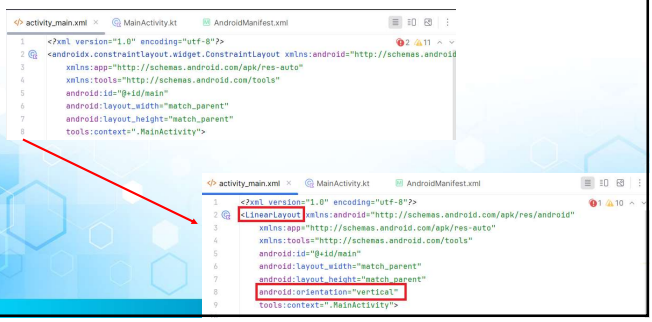


- Add attributes if it does not display
- Button LinearLayout by Drag and Drop

Layout



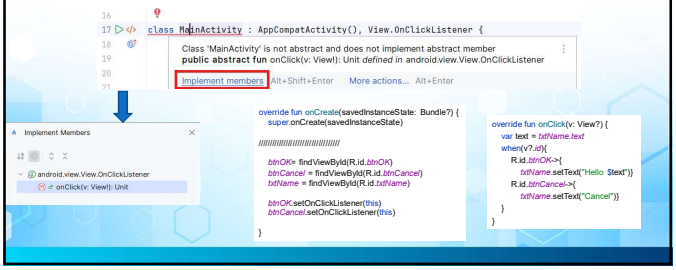
Layout



Click event

```
lateinit var btnOK:Button;
lateinit var btnCancel:Button;
lateinit var  txtName:EditText;
```

```
class MainActivity : AppCompatActivity(), View.OnClickListener {
```



Click event (2nd method)

```
override fun onCreate(savedInstanceState: Bundle?) {
   super.onCreate(savedInstanceState)

//////////////////////////////

   btnOK= findViewById(R.id.btnOK)
   btnCancel = findViewById(R.id.btnCancel)
   txtName = findViewById(R.id.txtName)

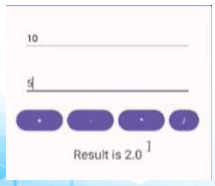
   btnOK.setOnClickListener(this)

btnOK.setOnClickListener {
      var text = txtName.text
      txtName.setText("Hello $text")}
}

   btnCancel.setOnClickListener(this)

}
```

Experimentals




```
var text = txtName.text.toString().toDouble()
```

Variable

- Keyword: ***var***

```
var variableName = value
val variableName = value
```

```
val myNum: Int = 5                  // Int
val myDoubleNum: Double = 5.99      // Double
val myLetter: Char = 'D'            // Char
val myBoolean: Boolean = true       // Boolean
val myText: String = "Hello"        // String
```

```
var a = 5
var name: String = "John" // String
var text = txtName.text.toString().toDouble()
```

- String

```
NameVariable.substring(startIndex, endIndex)
NameVariable.toUpperCase()
```

```
val name = "Hello world"
txtName.setText(name[3].toString())
```

Arrays

```
var Cars = arrayOf("Volvo", "Huyndai", "Toyota")
NameArray.set(index,value)        Cars[1]="Kia"
NameArray.get(index)              txtName.setText(Cars.size.toString())
val Num = Array<Int>(3) { 2;6;1 }
NameArray.joinToString()
```

```
when(v?.id){
    R.id.btnOK->{
        var Cars = arrayOf("Volvo", "Huyndai", "Toyota")
        Cars.set(1,"KIA")
        txtName.setText(Cars.joinToString())
    }
```

Conditions

```
var a=5
var b=7

var max = a
if (a < b) max = b


if (a > b) {
    max = a
} else {
    max = b
}
```

```
var a=5
var b=7

// As expression
var max = if (a > b) a else b

val maxLimit = 1
val maxOrLimit = if (maxLimit > a) maxLimit else if (a > b) a else b
```

```
val x = 2
when (x) {
    1 -> txtName.setText("x == 1")
    2 -> txtName.setText("x == 2")
    else -> txtName.setText("x is neither 1 nor 2")
}
```

Loop

```
var Cars = arrayOf("Volvo", "Huyndai", "Toyota")

for (i in 1..3) {
    print(i)
}
for (i in 6 downTo 0 step 2) {
    print(i)
}
for (i in Cars.indices) {
    txtName.setText(Cars[i])
}
```

```
var x=10
while (x > 0) {
    x--
}

var y=10;
do {
    y--
} while (y != 0)
```

```
var s=""
checkpoint@ for (i in Cars.indices)  {
    if(Cars.get(i)=="Huyndai") break@checkpoint
    s +=Cars.get(i)
}
txtName.setText(s)
```

```
var s=""
checkpoint@ for (i in Cars.indices)  {
    if(Cars.get(i)=="Huyndai") continue@checkpoint
    s +=Cars.get(i)
}
txtName.setText(s)
```

Functions

```
fun test(a: Int,b: Int)= a+b

fun test_1(a: Int,b: Int): Int{
    return  a+b
}

txtName.setText(test(2,5).toString())
```

```
fun test(a:Int, b:Int) = a+b

fun test1(a:Int, b:Int) :Int{
    return a+b
}

override fun onClick(v: View?) {
    var text = txtName.text
    when(v?.id){
        R.id.btnOK->{
            txtName.setText(test1( a: 5, b: 2).toString())
        }
```

OOP

```
class Car(var brand: String, var model: String, var year: Int)
{
}

val c1 = Car("Ford", "Mustang", 1969)
```

```
class Car(var brand: String, var model: String, var year: Int)
{
    fun GetBrand()=brand
}

class MainActivity : AppCompatActivity() , View.OnClickListener {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
```

```
when(v?.id){
    R.id.btnOK->{
        val c1 = Car( brand: "Ford",  model: "Mustang",  year: 1969)
        txtName.setText(c1.GetBrand())

    }
```

OOP

```
class Car(var brand: String, var model: String, var year: Int)
{
}

val c1 = Car("Ford", "Mustang", 1969)
```

- ***Inheritance***

```
// Superclass
open class MyParentClass {
    val x = 5
}

// Subclass
class MyChildClass: MyParentClass() {
    fun myFunction() {
        println(x) // x is now inherited from the superclass
    }
}
```

```
val myObj = MyChildClass()

myObj.myFunction()
```